とあるコンサルタントのつぶやき

MCS （Microsoft Consulting Services） の某コンサルタントがまったり語るテクノロジのお話です。

スレッドとオブジェクトインスタンス

Rate this article ★★★★★

nakama 12月 17, 2008

Share 8　0　0　15

さてさて、この blog、基本的には玄人向けに濃い話題ばっかり書いていたのですが、実は最近、新人さんの方々にも割と読んでいただいていることが判明。ちょ、さすがにぜんぜん意味不明なエントリが多いのでは？？ とか思ったりするのですが、そんな感じだとたまには割と入門者向けのエントリも書いてみたくなります。そんなわけで、今日はたまには基本に立ち返って、スレッドとオブジェクトインスタンスの話題を書いてみます。このネタを取り上げようと思ったのは、ある人からの質問メール。（ちょっと微修正しましたが^^）

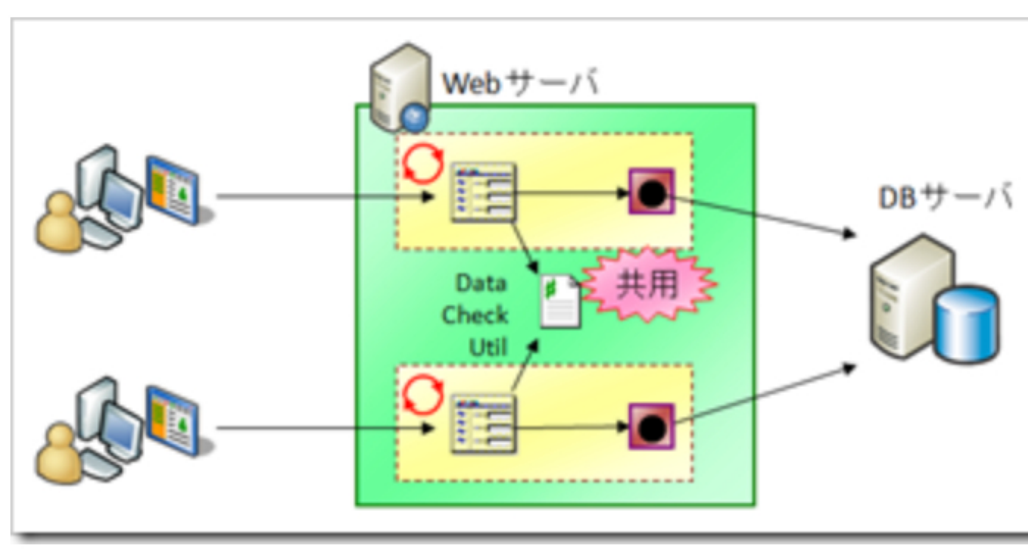
> 「最近、3 階層のアプリケーションを、課題で作っていたんですよ。
> そして、その UI 層の入力チェックメソッドを、
> メンバメソッドにすべきか static メソッドにするかを考えていて、
> ふと気になったことがあります。
>
> static メソッドだと、メンバメソッドと違って、毎回インスタンスを
> 生成する必要がなく、便利ですが、物理的に一つしかないので、
> 二つのスレッドから微小時間ずれてアクセスされると、トラブルが
> 起こるんではないでしょうか？」

これは**非常によい視点**、だと思います。というのもアプリケーション開発者にとって重要なのは、**どうすれば動くかではなく、どうするのが正しいのか**、だからです。単純に動くかどうか、だけでいえばメンバメソッドだろうと static メソッドだろうとどっちで書いても動く**ように見える**はずですが、過負荷状態になったときにもしかしたらトラブルになるかも？ だったらどう書くのが正しいのか？ ……というように論理的に考えることは非常に重要です。というわけで、この質問に敬意を表して、全力で答えてみたいと思います^^。

この質問、もうちょっと具体的な例として書いてみると、例えば UI 層のチェックメソッドを以下のように static メソッドとして実装した場合に

```
public class DataCheckUtil
{
    public static bool CheckEmailString(string email)
    {
        if (Regex.IsMatch(email, @"\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*") == true)
        {
            return true;
        }
        else
        {
            return false;
        }
```

これが Web アプリケーションのようなマルチスレッドアプリケーションで、複数スレッドから同時に呼び出された場合にちゃんと問題なく動くかどうか？ という質問です。


Webサーバ
DBサーバ
Data Check Util
共用

結論から先に書くと、このような処理は問題なく動作します。なぜ DataCheckUtil クラス内の static メソッドが同時に複数スレッドから呼び出されても問題なく動作するのかというと、それは、**実際には上図のように複数スレッドから同一のオブジェクトが使われるわけではない**からです。（つまり、簡単にいえば**上の図は正確なイラストではない**のですね^^）

このことを説明するために、.NET アプリケーションにおけるメモリ管理の話を少ししたいと思います。（ちなみに以下の話は Java などでもほぼ同じです。）

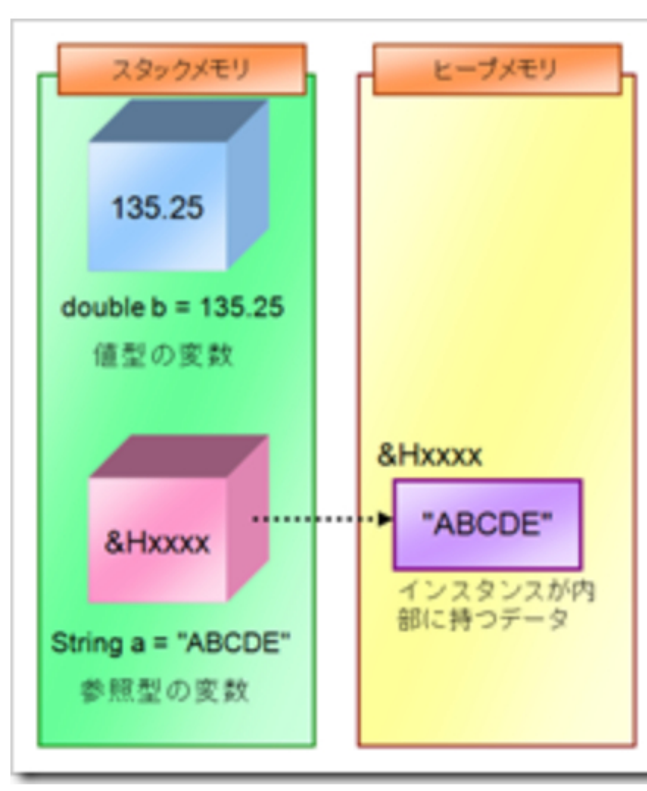
［スタックメモリとヒープメモリ］

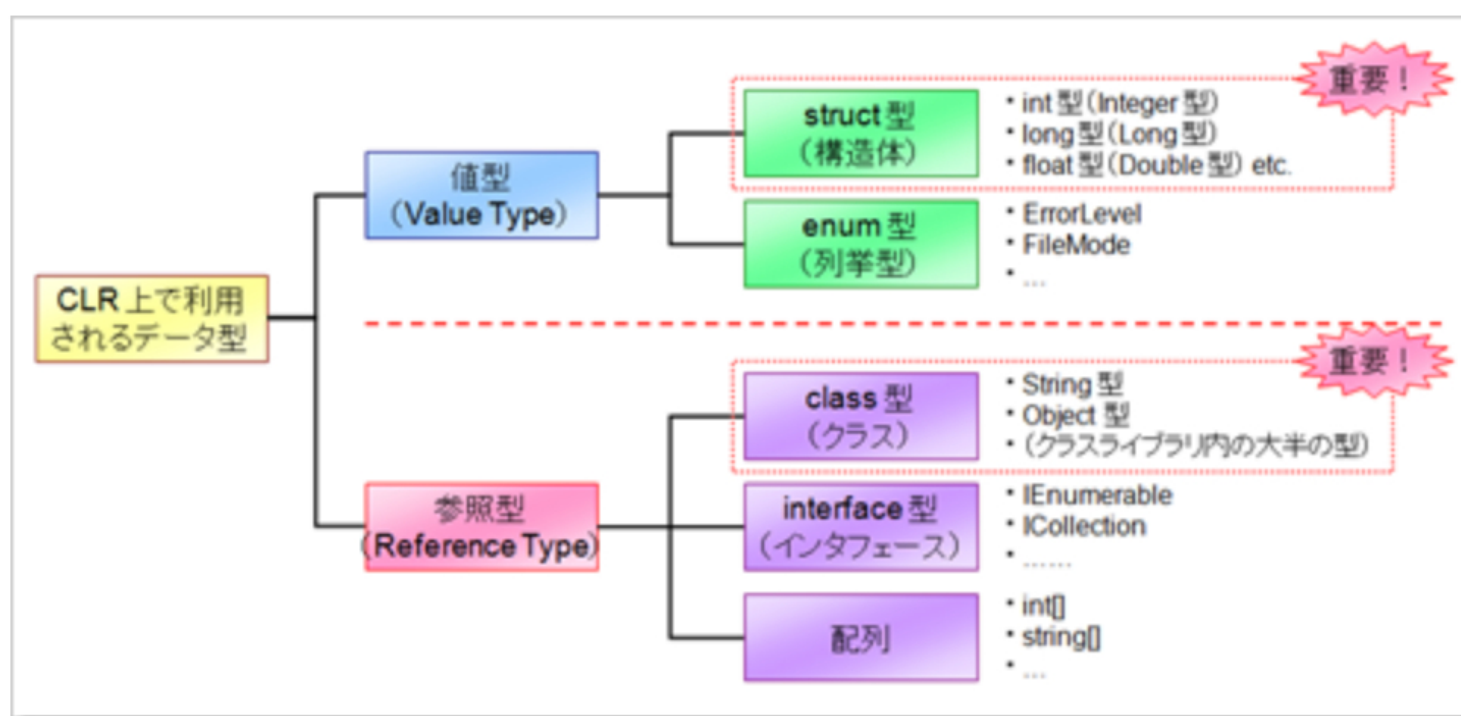
.NET や Java では、データをメモリに格納する際、スタックメモリとヒープメモリと呼ばれる 2 種類のメモリをうまく使い分けます。

- スタックメモリ（ローカルデータ変数を保持するための空間。容量が比較的小さい。）
- ヒープメモリ（参照型のデータのインスタンスを保持するための空間。容量が非常に大きい。）

.NET では、データ変数の型は、値型と参照型に大別されます。値型のデータ変数（例えば int 型や double 型のデータ変数）を宣言した場合、その変数領域はスタックメモリ内に確保され、そこに実際のデータが格納されます。しかし、参照型のデータ変数（例えば string 型など）を宣言した場合には、実体はヒープメモリに格納され、変数の箱にはそのヒープメモリ内のアドレスのみが保存される、という形になります。

```
class Program
{
    static void Main(string[] args)
    {
        double b = 135.25;
        string a = "ABCDE";
    }
}
```


スタックメモリ
ヒープメモリ
135.25
double b = 135.25
値型の変数
&Hxxxx
&Hxxxx
"ABCDE"
インスタンスが内部に持つデータ
String a = "ABCDE"
参照型の変数


CLR 上で利用されるデータ型
値型 (Value Type)
struct 型 (構造体)
・int 型 (Integer 型)
・long 型 (Long 型)
・float 型 (Double 型) etc.
enum 型 (列挙型)
・ErrorLevel
・FileMode
・…
重要！
参照型 (Reference Type)
class 型 (クラス)
・String 型
・Object 型
・(クラスライブラリ内の大半の型)
重要！
interface 型 (インタフェース)
・IEnumerable
・ICollection
・……
配列
・int[]
・string[]
・…

このような対策がなされているのは単純な理由で、

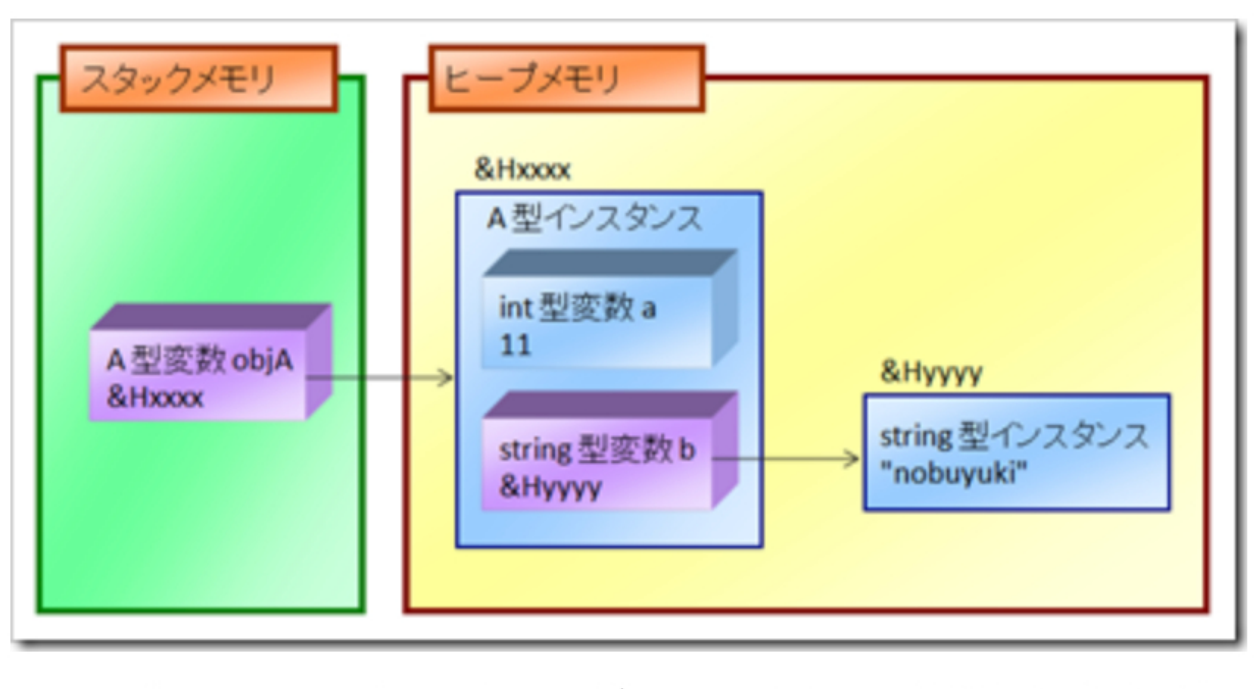
- スタックメモリは容量が少ない。
- double 型のような小さなデータ（数バイト程度）を保持することは問題がない。
- string 型のように巨大なデータ（文字列は場合によっては数 M バイトに膨れ上がることもある）を保持すると、あっという間にメモリ不足を起こしてしまう。

というわけです。

［クラスのインスタンスを作成する場合］

さて、.NET や Java では、クラスは参照型になります。このため、以下のようなコードを書いて実行すると、メモリイメージは下図のようになります。

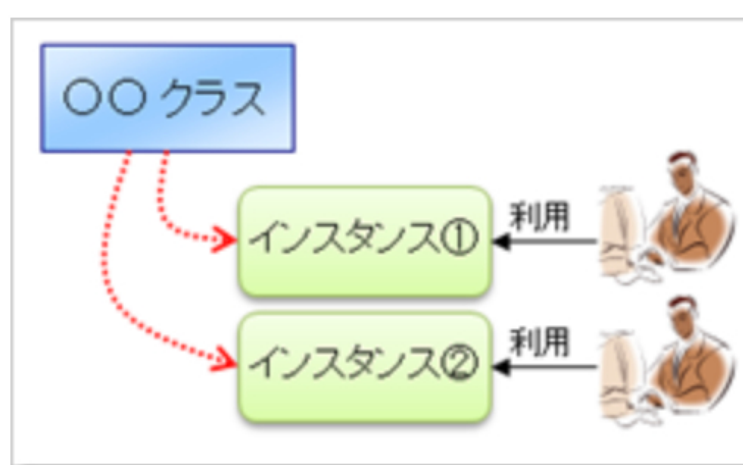
```
class Program
{
    static void Main(string[] args)
    {
        A objA = new A();
        objA.MethodX();
        objA.MethodY();
    }
}

public class A
{
```


スタックメモリ
ヒープメモリ
A 型変数 objA
&Hxxxx
&Hxxxx
A 型インスタンス
int 型変数 a
11
string 型変数 b
&Hyyyy
&Hyyyy
string 型インスタンス
"nobuyuki"

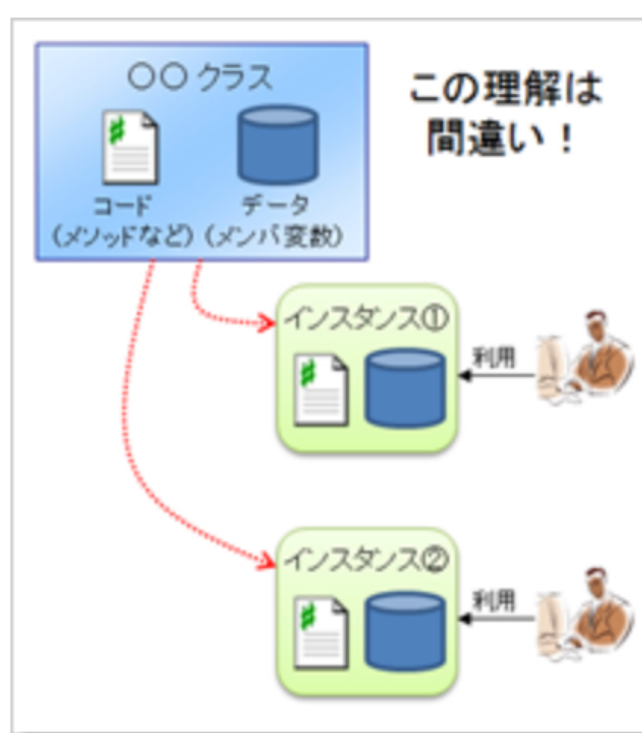
- クラス A のインスタンスが、ヒープメモリ空間内に作成される。
- ヒープメモリ空間内のオブジェクトインスタンス内には、オブジェクトの内部データである int 型変数 a と string 型変数 b のデータ格納領域ができる。
- int 型変数 a は値型なので、実体がそのまま入る。string 型変数 b は参照型なので、アドレス値が格納される。

さて、ここまでがいわゆる入門者向けの説明なのですが、実はここで初心者がハマるよくある誤解が、**インスタンスのところにコードがコピーされて実行される**、というものです。

一般的に、クラスとインスタンスの関係は、鯛焼きの型と鯛焼きの関係だと言われます。（分かりにくいような気もしますが^^）


○○ クラス
インスタンス①
インスタンス②
利用
利用

ここで注意しなければならないのは、**クラスはアプリケーション実行コードを含んでいるが、インスタンスは実行コードを含んではいない**、という点です。つまり、下図のような理解は間違いです。


○○ クラス
コード (メソッドなど)
データ (メンバ変数)
この理解は間違い！
インスタンス①
インスタンス②
利用
利用

実は、**ヒープメモリに作成されるインスタンスには、実行コードが含まれておらず、データ変数のみが格納されています。**アプリケーション実行コードは一か所にのみ存在し、インスタンスごとには保持されていません。

［スタックメモリとコールスタック］

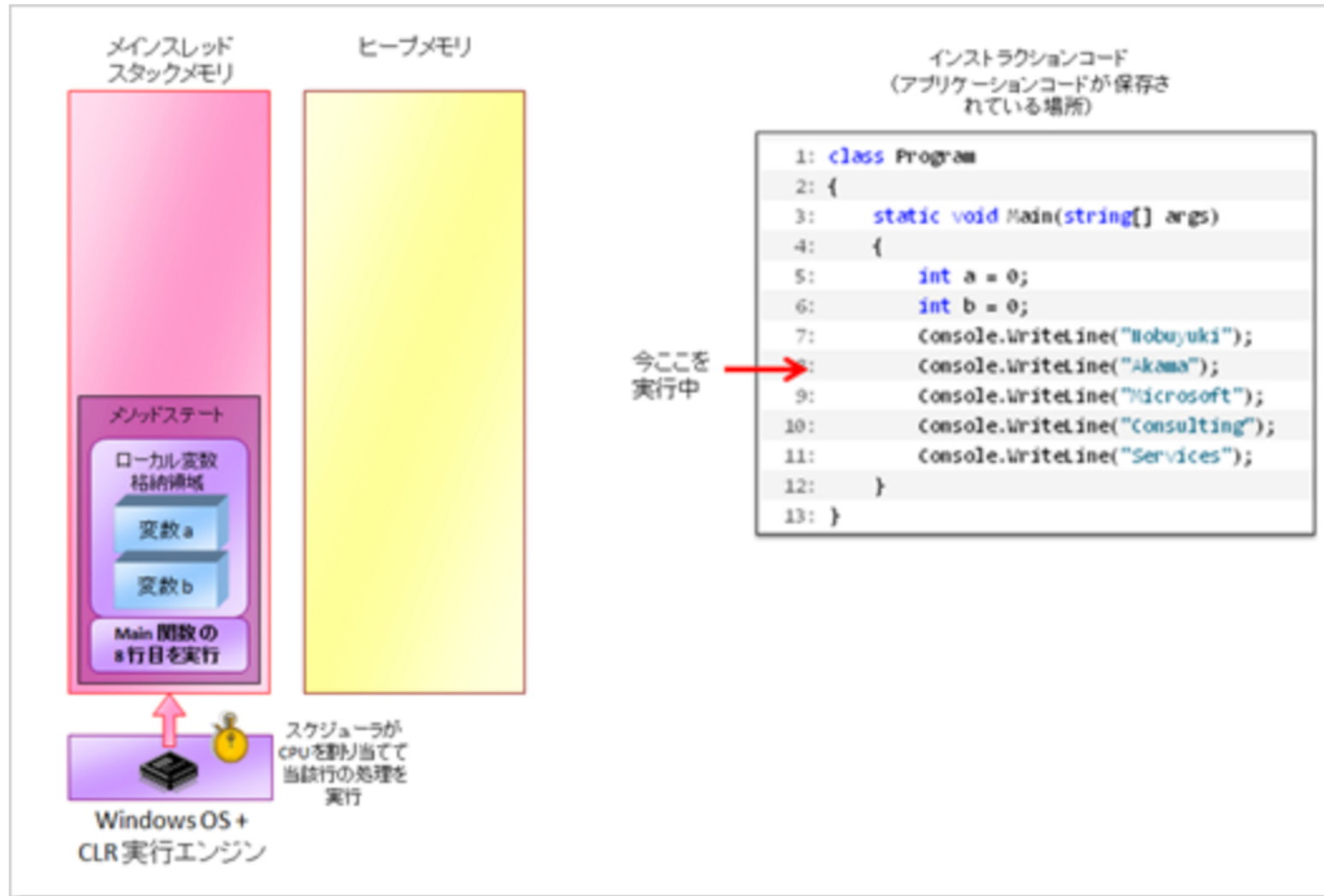
ではなぜ実行コードがコピーされなくても正しく処理できるのか？ これを説明するためには、もう一つ、**スタックメモリとコールスタック**について解説する必要があります。

そもそもなぜスタックメモリは**「スタックメモリ」**と呼ばれているのか？ それは、このスタックメモリというのが**コールスタック**を管理しているものだからです。コールスタックとインストラクションコードを理解するために、まず以下のようなサンプルコードを取り上げてみましょう。

```
class Program
{
    static void Main(string[] args)
    {
        int a = 0;
        int b = 0;
        Console.WriteLine("Nobuyuki");
        Console.WriteLine("Akama");
        Console.WriteLine("Microsoft");
        Console.WriteLine("Consulting");
        Console.WriteLine("Services");
    }
```

このようなアプリケーションは、次のように実行されます。

- アプリケーションコードは、メモリ内の特定領域（ヒープでもスタックでもないところ）に保持されている。
- Main() 関数が動作すると、**メインスレッドに対してスタックメモリ領域が作成され**、このスタックメモリ内に、現在実行している行番号と、ローカル変数が保持される。
- このスレッドに CPU が割り当てられると、CPU が当該行のコードを実行し、次の行へとどんどん進んでいく。（ちなみに現在の実行行番号のことを、**インストラクションポインタ**と呼びます。）
- 結果として、インストラクションポインタ（図中の→マーク）が順次下方向に動いていき、そこに書かれている処理命令が、**スタックメモリ内のデータに対して実施される**、という形になります。


メインスレッド スタックメモリ
ヒープメモリ
インストラクションコード (アプリケーションコードが保存されている場所)
メソッドステート
ローカル変数 格納領域
変数 a
変数 b
Main 関数の 8 行目を実行
今ここを実行中
スケジューラが CPU を割り当てて当該行の処理を実行
Windows OS + CLR 実行エンジン

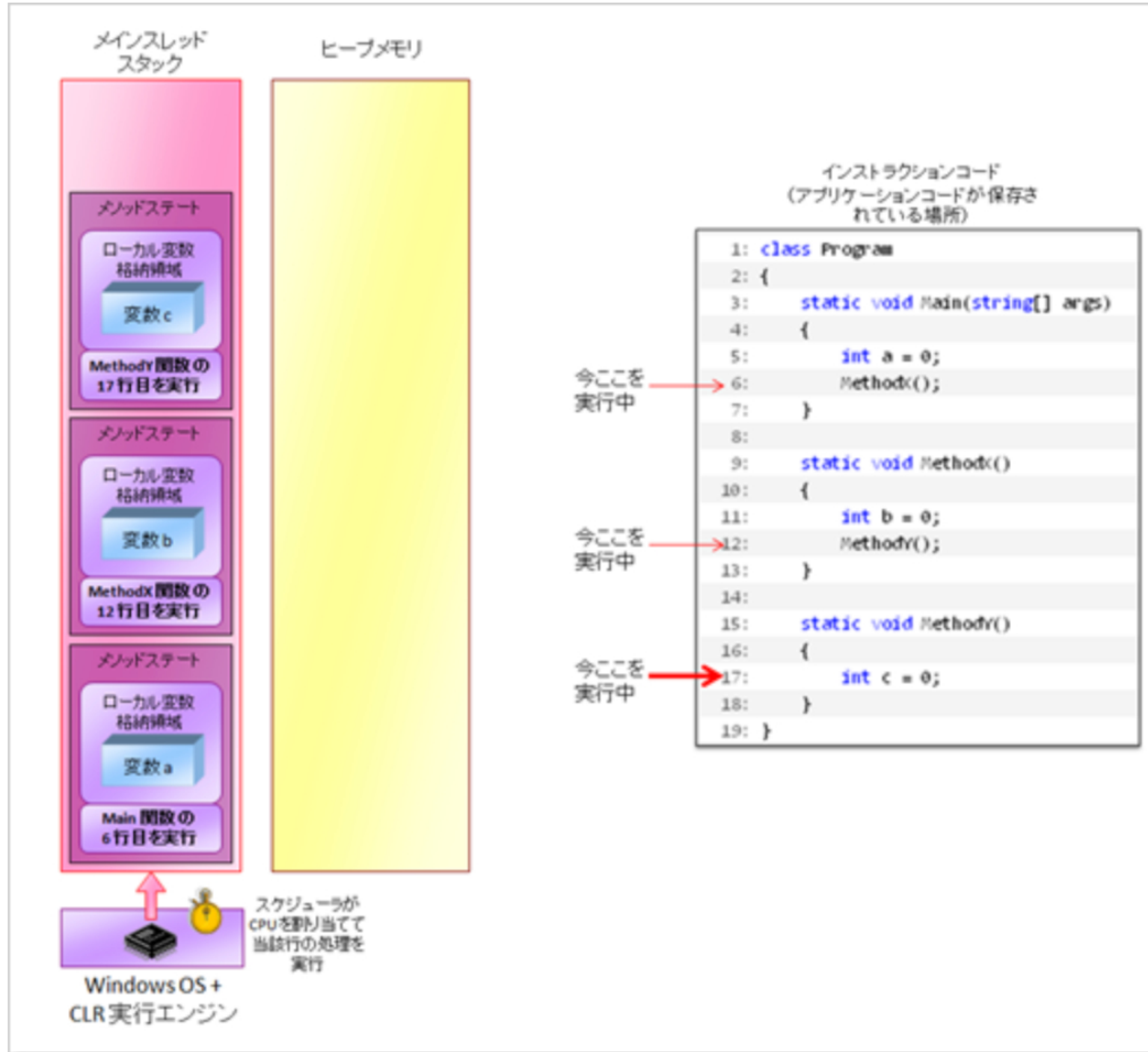
```
1: class Program
2: {
3:     static void Main(string[] args)
4:     {
5:         int a = 0;
6:         int b = 0;
7:         Console.WriteLine("Nobuyuki");
8:         Console.WriteLine("Akama");
9:         Console.WriteLine("Microsoft");
10:        Console.WriteLine("Consulting");
11:        Console.WriteLine("Services");
12:    }
13: }
```

では今度は、あるメソッドから別のメソッドが呼び出される例として、次のようなものを考えてみます。

```
class Program
{
    static void Main(string[] args)
    {
        int a = 0;
        MethodX();
    }

    static void MethodX()
    {
        int b = 0;
        MethodY();
```

この場合、アプリケーションは Main → MethodX() → MethodY() とネストして呼び出されていくことになるのですが、この場合は下図のような動きをします。

- まず、Main() 関数が動いていくと、実行行が 3, 4, 5, 6 行目と進んでいく。 これにより、**インストラクションコードの処理が、スタックメモリ内のデータに対して実施される。**
- 6 行目まで達すると、MethodX() が呼び出される。すると、スタックメモリ内にもう一つ、メソッドステートが**重ねられ**、この中身が 9, 10, 11, 12 行目と進んでいく。
- 12 行目まで達すると、MethodY() が呼び出される。すると．．．（以下略）
- そしてメソッドが終了すると、メソッドステートが消されて元に戻っていく。（このようにスタックメモリにはメソッドの処理状態が**積み重ねられていく**ために、「スタックメモリ」と呼ばれている）


メインスレッド スタック
ヒープメモリ
インストラクションコード (アプリケーションコードが保存されている場所)
メソッドステート
ローカル変数 格納領域
変数 c
MethodY 関数の 17 行目を実行
メソッドステート
ローカル変数 格納領域
変数 b
MethodX 関数の 12 行目を実行
メソッドステート
ローカル変数 格納領域
変数 a
Main 関数の 6 行目を実行
今ここを実行中
今ここを実行中
今ここを実行中
スケジューラが CPU を割り当てて当該行の処理を実行
Windows OS + CLR 実行エンジン

```
1: class Program
2: {
3:     static void Main(string[] args)
4:     {
5:         int a = 0;
6:         MethodX();
7:     }
8:
9:     static void MethodX()
10:    {
11:        int b = 0;
12:        MethodY();
13:    }
14:
15:    static void MethodY()
16:    {
17:        int c = 0;
18:    }
19: }
```

つまり、ここまでの話からわかるように、

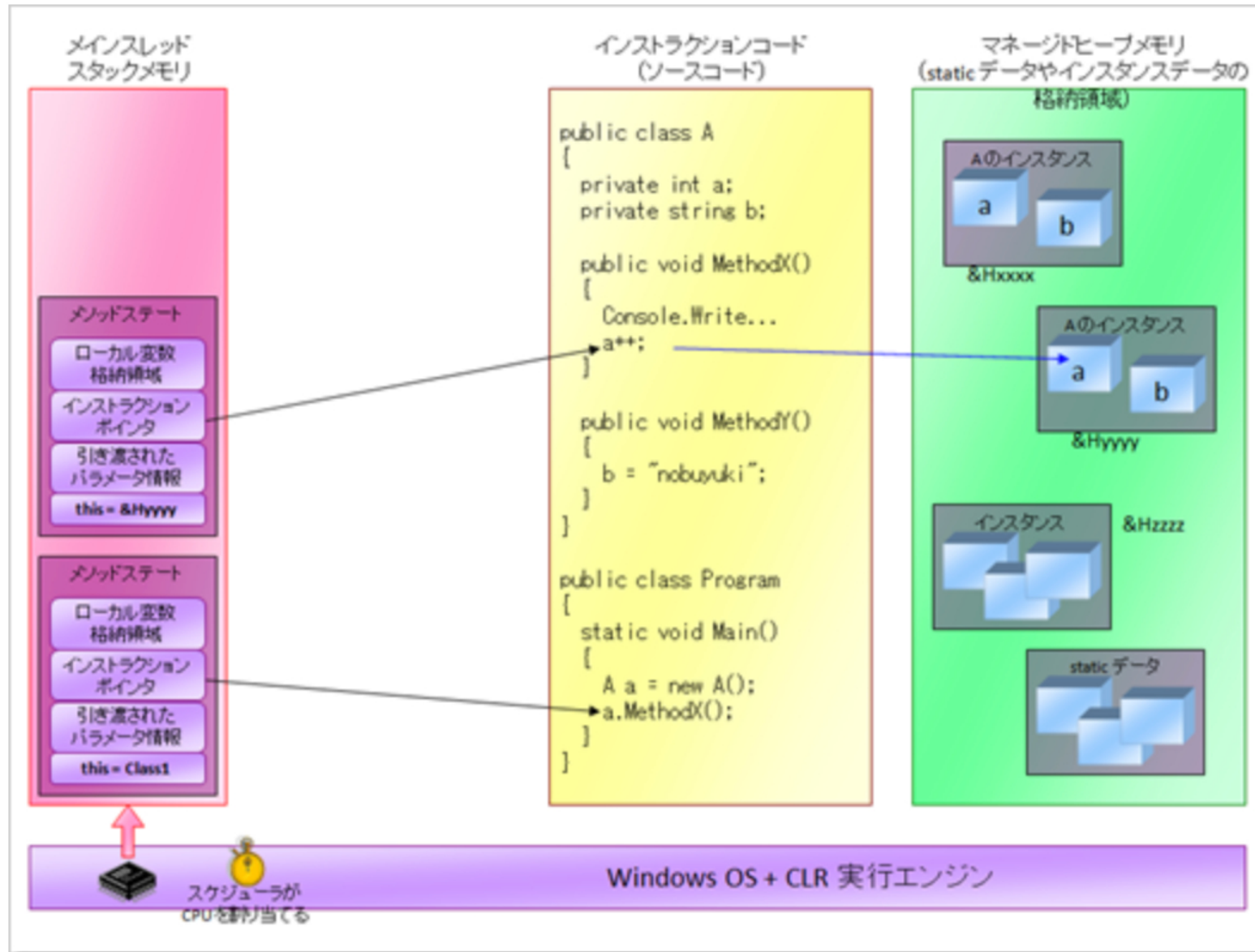
- 実行コードとデータ変数領域は、物理的にはわかれている。
- 実行コードに書かれている処理が、スタックメモリやヒープメモリ内におかれたデータに対して実施される。

という形になっています。（＝コードが物理的に各スレッドやインスタンスにコピーされて動いているわけではない）

［オブジェクトインスタンスがかかわる場合のコールスタック］

さて、上図はオブジェクトインスタンスがかかわらない場合のコールスタックですが、ではオブジェクトインスタンスがかかわるとどのようになるのか？ 具体例を示すと、次のようになります。（ちょっと分かりにくいと思いますので、しっかり図とコードを見てください。自分で一度、イラストを描いてみるのもよいと思います。）

```
public class A
{
  private int a;
  private string b;

  public void MethodX()
  {
    Console.Write...
    a++;
  }

  public void MethodY()
```


メインスレッド スタックメモリ
インストラクションコード (ソースコード)
マネージドヒープメモリ (static データやインスタンスデータの格納領域)
メソッドステート
ローカル変数 格納領域
インストラクション ポインタ
引き渡された パラメータ情報
this = &Hyyyy
メソッドステート
ローカル変数 格納領域
インストラクション ポインタ
引き渡された パラメータ情報
this = Class1
A のインスタンス
a
b
&Hxxxx
A のインスタンス
a
b
&Hyyyy
インスタンス
&Hzzzz
static データ
スケジューラが CPU を割り当てる
Windows OS + CLR 実行エンジン

```
public class A
[
  private int a;
  private string b;

  public void MethodX()
  [
    Console.Write...
    a++;
  ]

  public void MethodY()
  [
    b = "nobuyuki";
  ]
]

public class Program
[
  static void Main()
  [
    A a = new A();
    a.MethodX();
  ]
]
```

キーポイントを示すと、以下のようになります。

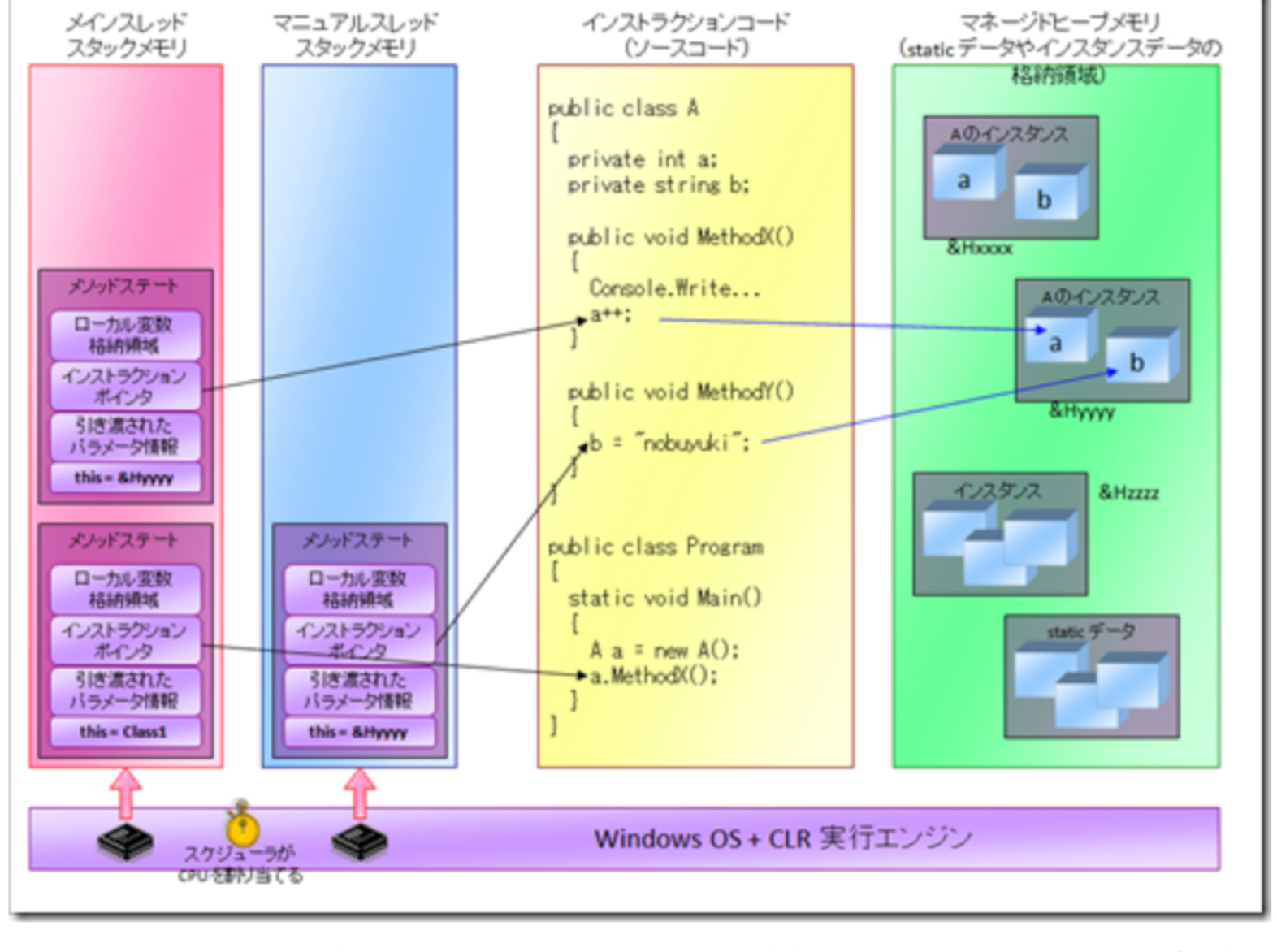
- メソッドステートの中に、「今、どのインスタンスを処理しているのか？」のデータが含まれるようになる。一方、インストラクションポインタ（＝現在の実行行番号）は、あくまで行番号を示している。
- 実際の処理は、インストラクションポインタと、処理対象インスタンスデータの二つによって行われる。例えば上図の場合、"a++"の処理は、現在の処理対象インスタンスである &Hyyyy のインスタンスに対して行われることになる。

［マルチスレッドアプリにおけるスタックメモリとヒープメモリの図］

さて、以上はシングルスレッドアプリケーションの場合について解説しましたが、マルチスレッドアプリケーションの場合には、

- **スタックメモリは、スレッドごとに作成される。**
- **ヒープメモリは、複数のスレッド間で共有される。**
- **インストラクションコードは、複数のスレッド間で共有される。**

という動きをします。よって、例えばメインスレッド以外にマニュアルスレッドが動作しているようなマルチスレッドアプリのメモリ構造は、下図のようなイメージなります。


メインスレッド スタックメモリ
マニュアルスレッド スタックメモリ
インストラクションコード (ソースコード)
マネージドヒープメモリ (static データやインスタンスデータの格納領域)
メソッドステート
ローカル変数 格納領域
インストラクション ポインタ
引き渡された パラメータ情報
this = &Hyyyy
メソッドステート
ローカル変数 格納領域
インストラクション ポインタ
引き渡された パラメータ情報
this = Class1
メソッドステート
ローカル変数 格納領域
インストラクション ポインタ
引き渡された パラメータ情報
this = &Hyyyy
A のインスタンス
a
b
&Hxxxx
A のインスタンス
a
b
&Hyyyy
インスタンス
&Hzzzz
static データ
スケジューラが CPU を割り当てる
Windows OS + CLR 実行エンジン

```
public class A
[
  private int a;
  private string b;

  public void MethodX()
  [
    Console.Write...
    a++;
  ]

  public void MethodY()
  [
    b = "nobuyuki";
  ]
]

public class Program
[
  static void Main()
  [
    A a = new A();
    a.MethodX();
  ]
]
```

ではここで、Web アプリケーションを例にとって考えてみましょう。たとえば、以下のようなコードを持つ A.aspx ファイルがマルチスレッドで実行された場合を考えてみます。この場合にどんなメモリ配置になるのかを描いてみてください。

```
public partial class _Default : System.Web.UI.Page
{
    private int a = 0;
    static protected int c = 0;
```

Follow Us

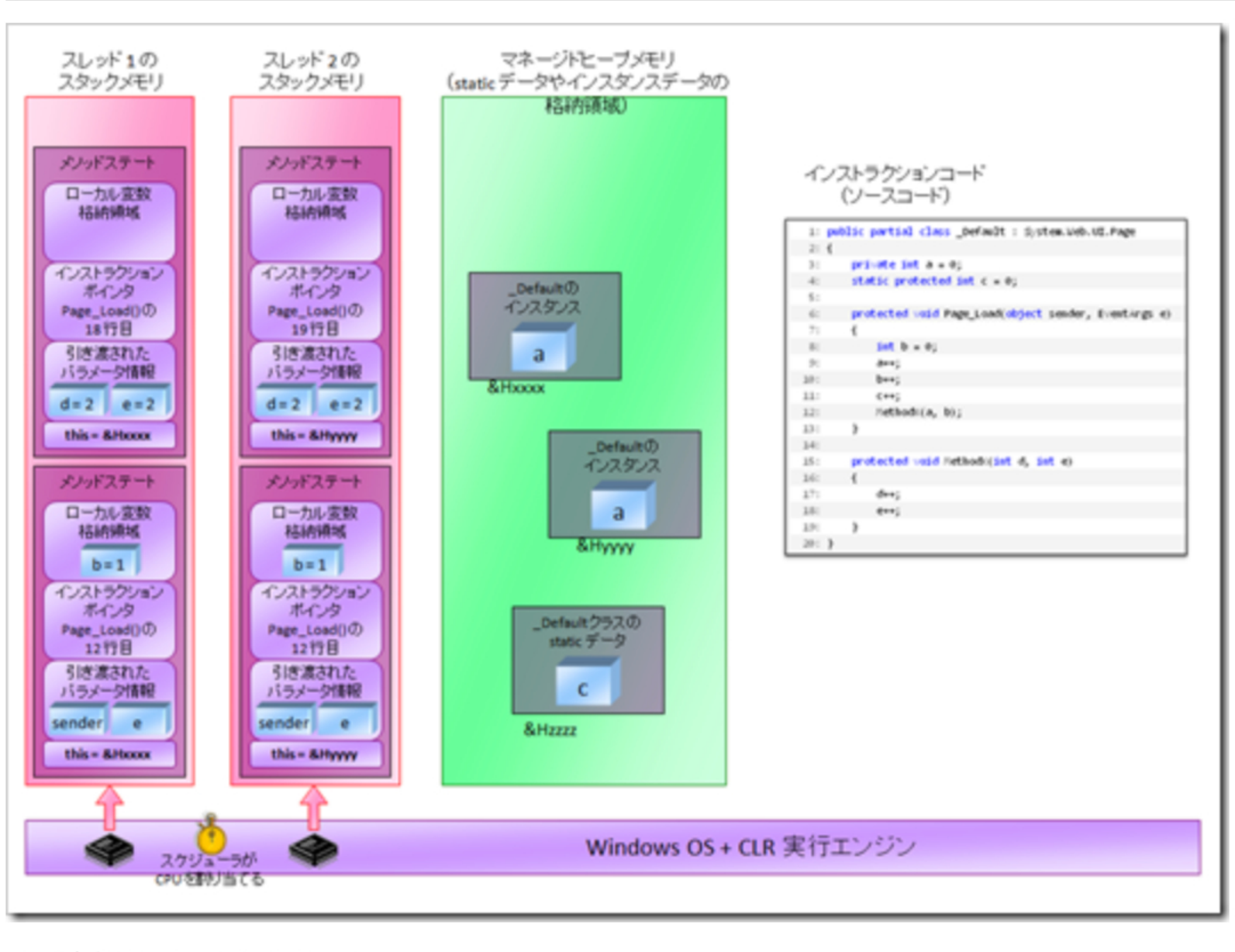
Popular Tags

Basics　Azure　WindowsForms　ASP.NET　Silverlight　64bit　WCF　Skills　WPF　Transaction　WCS　IE　Pages

Archives

- 2018年11月 (1)
- 2018年9月 (1)
- 2018年8月 (2)
- 2018年6月 (1)
- 2018年2月 (1)
- 2017年6月 (1)
- 2017年5月 (1)
- 2016年8月 (11)
- 2016年7月 (9)
- 2016年6月 (2)
- All of 2018 (6)
- All of 2017 (2)
- All of 2016 (22)
- All of 2014 (1)
- All of 2012 (5)
- All of 2011 (6)
- All of 2010 (14)
- All of 2009 (15)
- All of 2008 (26)

```
 5:
 6:     protected void Page_Load(object sender, EventArgs e)
 7:     {
 8:         int b = 0;
 9:         a++;
10:         b++;
11:         c++;
12:         MethodX(a, b);
```


スレッド 1 の
スタックメモリ
スレッド 2 の
スタックメモリ
マネージドヒープメモリ
(static データやインスタンスデータの格納領域)
メソッドステート
ローカル変数格納領域
インストラクションポインタ Page_Load()の 18行目
インストラクションポインタ Page_Load()の 19行目
引き渡されたパラメータ情報
d = 2
e = 2
this = &Hxxxx
this = &Hyyyy
b = 1
インストラクションポインタ Page_Load()の 12行目
sender
e
_Defaultの インスタンス
a
&Hxxxx
&Hyyyy
_Defaultクラスの static データ
c
&Hzzzz
インストラクションコード
(ソースコード)
スケジューラが CPU を割り当てる
Windows OS + CLR 実行エンジン

この例からわかるように、

- メソッド内部で宣言されるローカル変数やメソッド引数については、スレッドごとにデータ変数領域が作られる。
- しかし、static 宣言されたデータ変数については、全体で一つしかない。

という形になっています。このため、実は上記のコードの場合、**データ変数 c を操作する 11 行目の処理を 2 つ以上のスレッドが同時に処理した場合にトラブルが起こる（ロストアップデートなど）可能性があります。**上記のコードの場合には、11 行目の c++ というコードを複数スレッドが同時に実行しないようにするために、以下のようなスレッド間同期制御コードを書かなければなりません。

```
protected void Page_Load(object sender, EventArgs e)
{
    int b = 0;
    a++;
    b++;
    lock (typeof(_Default))
    {
        c++;
    }
    MethodX(a, b);
}
```

※ lock 構文についてはここでは説明しませんので、興味がある方は書籍などで調べてみてください。

さて、ここまでの話を統合して、最初の質問に戻りましょう。例えば UI 層のチェックメソッドを以下のように static メソッドとして実装した場合を考えてみます。

```
public class DataCheckUtil
{
    public static bool CheckEmailString(string email)
    {
        if (Regex.IsMatch(email, @"\w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*") == true)
        {
            return true;
        }
        else
        {
            return false;
        }
```

これが Web アプリケーションのようなマルチスレッドアプリケーションで複数スレッドから同時に呼び出されたとして問題があるのか？ ……もう答えは明らかですね。マルチスレッドアプリケーションで複数スレッドからこのメソッドが呼び出されても、全く問題がありません。

［スレッドセーフなメソッド］

以上のことから、次のようなことが言えます。

- static データ変数などを（lock 処理などをせずに）無造作に操作しているメンバメソッドは、マルチスレッド動作させた場合にトラブルが出る危険性がある。
- ローカル変数しか操作していないようなメソッドは、マルチスレッド動作せても問題がない。（※ ローカル変数が参照型で、そのオブジェクトインスタンスを別スレッドからも操作しせてたらダメですが；）

一般に、マルチスレッドアプリケーションにおいて複数スレッドから同時に呼び出されたとしても問題が発生しないメソッドのことを、**スレッドセーフなメソッド**と呼びます。Java や .NET の基本的なクラスライブラリの多くはスレッドセーフに設計されていますが、**すべてがスレッドセーフとは限りません。**このため、安易にマルチスレッド動作アプリケーション（例えば Web アプリケーションや Web サービス）からクラスライブラリを使うと、トラブルが起こる危険性があるわけです。

とはいえ、.NET の中でも我々が普段使うようなメソッドについてはほとんどがスレッドセーフに作られています。例えば Int32 型のドキュメントを見てみると、**すべてのメンバがスレッドセーフに作られている**、と明記されています。このため、例えば Int32.Parse() メソッドなどを無造作に呼び出しても、トラブルは発生しません。

［最後にクイズ］

では、最後にクイズを出しておきたいと思います。以下に示す クラス X の 3 つのメソッドは、それぞれスレッドセーフでしょうか？

```
public class X
{
    private int a;
    static private int b;

    public void MethodA(int c)
    {
        string d = "Nobuyuki";
        c++;
        d = "Akama";
        Console.WriteLine(c);
        Console.WriteLine(d);
```

正解は……ないしょです。（え？w） みなさんで考えてみてくださいね。

［まとめ］

というわけで、今回のエントリのまとめです。

- アプリケーションコードは、インスタンスごとやスレッドごとにコピーされて実行されるわけではない。
- スタックメモリは、スレッドごとに作成され、コールスタックが管理される。
- パラメータ引数やメソッド内部ローカル変数は、スタックメモリ内にデータ変数領域が作られる。
- ヒープメモリは、スレッド間で共有され、インスタンス内部データや static データなどが管理される。
- マルチスレッドアプリケーションで、複数スレッドから呼び出しても異常動作を起こさないメソッドを、スレッドセーフなメソッドと呼ぶ。
- .NET Framework のクラスライブラリは、すべてがスレッドセーフというわけではない。

スレッドとオブジェクトインスタンスの関係は、私もオブジェクト指向を学習し始めた当初に相当悩んだ部分で、このエントリでは偉そうに書いてますが；、ぶっちゃけホントにわからなかったです……。今回書いたような話は非常に基礎的なことでありながらも、「なんとなく」で済ませてしまっている方も多いのではないかと思いますが、正確な理解には欠かせない部分なので、ぜひ一度は自分で絵などを描いてみて、理解していただければと思います。

＃ ってむちゃくちゃ長いエントリになってしまった……

＃ 結構ちゃんと説明しようと思うと大変な話なんですよねぇ、この話。

Tags Basics

Comments (15)

kazenami
12月 23, 2008 9:44 am
ふぁ～

nakamaｻﾝ、いささか全力過ぎやしませんでしょうか（＋＿＋）

・・・

（嘘です。疑問がに思っていたことが、限りなくクリアに、理解されかかってきました。。）

と、いう冗談はさておき、

クイズしてみました！

私の答えは、

MethodA:スレッドセーフ

（ローカル変数c,dはスレッドごとに作成されるため）

MethodB:×

（データ変数bを操作する処理を2つ以上のスレッド）

MethodC:スレッドセーフ

kazenami
12月 23, 2008 10:11 am
まことに申し訳ありません。

操作を誤り、途中にもかかわらず、Submitしてしまいました。

ということで、再度書き込みさせていただきます。

MethodB:×

（データ変数bを操作する処理を、2つ以上のスレッドが同時に行ってしまうとロストアップデートの可能性があるため）

MethodC:スレッドセーフ

（自信がありません）

↑オブジェクトインスタンスが、どのような条件の時に

変わるかというのが、分からなかったためです。

クラスが変わると、オブジェクトインスタンスも変わるのは分かったのですが、スレッドが変わる時は、どうなのかが分かりません。

例の６個目（？）で、メインスレッドとマニュアルスレッドを使用している際は、スレッドが異なっていても、（同じクラスのため？）

オブジェクトインスタンスのアドレスは両方とも&Hyyyyになっています。

しかし、例の７個目では、

スレッド１のオブジェクトインスタンスのアドレス：&Hxxxx

スレッド2のオブジェクトインスタンスのアドレス：&Hｙｙｙｙ

となっています。

どのような条件で、オブジェクトインスタンスはつくり変わるのでしょうか。

あまりにもﾄﾝﾁﾝｶﾝな質問でなければ、教えていただけると嬉しいです。

（長文失礼しました。

何だか、先程からここのblog荒らしている気がしてならないのですが・・・・、いやしかし、ここは勇気をもってもう一度Submit（`・ω・´））

HashedBeef
12月 24, 2008 5:08 am
はじめまして。

実は今月のはじめくらいからマルチスレッドについて悩んでおりまして、

このページを発見したときには目からウロコでした。

大変参考になりました。

私も問題を解いてみたのですがまだ理解が足りないみたいで疑問がちらほらでてきました。

まずMethodAについてです。

「スレッドセーフなメソッド」のところに書いてある、

”ローカル変数しか操作していないようなメソッドは、マルチスレッド動作せても問題がない。（※ ローカル変数が参照型で、そのオブジェクトインスタンスを別スレッドからも操作しせてたらダメですが；）”

という別のスレッドからの操作というのが気になりました。

ローカル変数が参照型でもスレッドごとに作成した場合はヒープ領域が別々で作られるなら（ここがわかりません）、MethodAもスレッドセーフなのではないかと思います。

また、MethodCについては、aを使うメソッドが他にないのでスレッドセーフではあると思うのですが、仮にaを使うメソッドがあった場合は値が変わってしまうためスレッドセーフではないのではないかと。

また、クラスXのインスタンス生成後にマルチスレッドにするのか、スレッドごとにXのインスタンスを生成するかで、スレッドセーフかどうかはわかりませんが互いに干渉するかしないかは変わってくるのではないでしょうか。

私はスレッドごとに参照型の変数を作成するとヒープ領域が別でとられるのかどうかがわかりません。たぶん別だと思ってますが。

私も長々と書いてしまいまして申し訳ありませんm(＿＿)m

ぜひぜひご指導お願いします。

nakama
12月 24, 2008 12:47 pm
えと、お二人の質問を元にエントリを立てておきましたので、そちらも参照してください^^。

> kazenami さん

こんばんは。クイズの答えですが、MethodA, B については正解。

問題は MethodC ですが、これはオブジェクトインスタンスがどのように作られているか次第で安全か否かが変わってきます。これは、もう一つのエントリの方を参考にして考えてみてください。

> HashedBeef さん

マルチスレッドアプリケーションはなかなか難しいので、ぜひこの機会に勉強してください。

ちなみに .NET 関係ですと、"Essential .NET" という本が詳しいので、ここのエントリの内容が理解できた上でさらに学習を深めたい場合にはそちらを読んでいただくとよいと思います。

> まずMethodAについてです。

> 「スレッドセーフなメソッド」のところに書いてある、

> ”ローカル変数しか操作していないようなメソッドは、マルチスレッド動作せても問題がない。（※ ローカル変数が参照型で、そのオブジェクトインスタンスを別スレッドからも操作しせてたらダメですが；）”

> という別のスレッドからの操作というのが気になりました。

これは、例えば以下のような Web ページをマルチスレッド動作させた場合を考えてみるとわかりやすいと思います。

public partial _Default : Page

{

private static MyClassA objA = new MyClassA();

protected void Page_Load(object sender, EventArgs e)

{

MyClassA objX = objA;

objX.MethodZ();

}

}

この場合、objX はローカル変数（＝変数のハコはスレッドごとに作られる）ですが、実際に操作しているインスタンスは常に同じものになります。このため、この Web ページはマルチスレッド動作させた場合、（ローカル変数しか触っていないようなコードになっているけど）同一インスタンスを複数スレッドから操作することになる、というわけです。

> ローカル変数が参照型でもスレッドごとに作成した場合はヒープ領域が別々で作られるなら（ここがわかりません）、MethodAもスレッドセーフなのではないかと思います。

ローカル変数の「箱」はスレッドごとに作られますが、ヒープメモリ空間は、プロセス内にひとつ（正確には AppDomain 内にひとつ）です。よって、ローカル変数が参照型の場合には、

・ローカル変数の「箱」（アドレス値を入れる箱）は、スレッドごとにできる。

・でもインスタンスは、プロセス内の共有空間であるヒープ領域に作られる。

となります。

HashedBeef
12月 25, 2008 5:39 am
回答ありがとうございます！！

さらに別のエントリまで作っていただいて…

大変お手数をおかけしました＾＾；

細かいところを気にする性分なもので、Essential .NET、読んでみたいと思います。

私の疑問は続・スレッドとオブジェクトインスタンスのプールスレッドの図で大体解決したのですが、確認のため私なりにnakamaさんのまとめを編集してみました。

・ローカル変数の「箱」（アドレス値を入れる箱）は、スレッドごとにスタックに作成される。

・でもインスタンスは、プロセス内の共有空間であるヒープ領域に作られる。

そしてインスタンス生成について以下のように場合わけできる

○(newで)新しく作られる場合

→ 共有ヒープ領域内に、各スレッドで生成されるたびにそれぞれ独立な実体として保持される＝スレッドセーフ

○既存のインスタンスを新しく宣言したインスタンス変数に代入する場合

→ 共有ヒープ領域内に保持される実体は、同一の実体を指す＝スレッドセーフではない

こんな理解でいいでしょうか。

また、nakamaさんが記述されたコードの

private static MyClassA objA = new MyClassA();

のstaticの部分を取ると、

マルチスレッドでこのページが呼び出されてもスレッドセーフだと思われるのですがどうでしょうか!?

nakama
12月 27, 2008 2:11 am
> HashedBeef さん

いえいえ、エントリ読んでいただいてありがとうございます^^。

さてご質問の件ですが、

> ・ローカル変数の「箱」（アドレス値を入れる箱）は、スレッドごとにスタックに作成される。

> ・でもインスタンスは、プロセス内の共有空間であるヒープ領域に作られる。

はい、その通りです。

> そしてインスタンス生成について以下のように場合わけできる

> ○(newで)新しく作られる場合

> → 共有ヒープ領域内に、各スレッドで生成されるたびにそれぞれ独立な実体として保持される＝スレッドセーフ

えと、「共有ヒープ領域内に、各スレッドで生成されるたびにそれぞれ独立な実体として保持される」は正しいのですが、これをスレッドセーフとは呼びません。

> ○既存のインスタンスを新しく宣言したインスタンス変数に代入する場合

> → 共有ヒープ領域内に保持される実体は、同一の実体を指す＝スレッドセーフではない

こちらも同様で、「共有ヒープ領域内に保持される実体は、同一の実体を指す」は正しいですが、「スレッドセーフではない」とは言いません（し、スレッドセーフでないとは限りません）。

スレッドセーフという言葉は、「複数スレッドから同一インスタンスの同一メソッドを呼んでもトラブルが起きない」ことを指します。

端的に言えば、クラスやそのメソッドの「特性」みたいなものなのですね。

例えば一番最後の例の場合、「既存のインスタンスを新しく宣言したインスタンス変数に代入する場合には、共有ヒープ領域内に保持される実体は、同一の実体を指す」のは正しいのですが、そのインスタンスがたとえば string 型のインスタンスだった場合には、トラブルが起きません。なぜなら、string クラスのすべてのメソッドはスレッドセーフに作られているからです。

> また、nakamaさんが記述されたコードの

> private static MyClassA objA = new MyClassA();

> のstaticの部分を取ると、

> マルチスレッドでこのページが呼び出されてもスレッドセーフだと思われるのですがどうでしょうか!?

はい、こちらは正解です。

HashedBeef
12月 27, 2008 3:27 am
スレッドセーフについての認識が間違っていたようなので、もう少し勉強してみます。

これからもちょくちょく参考にさせていただきます。

ありがとうございました！

だどさん
12月 28, 2008 3:57 pm
丁寧な解説ありがとうございます。

表現の問題だけなのかわかりませんが、疑問に思った点だけ記載させていただきます。。。

「インストラクションポインタ（＝現在の実行行番号）は、あくまで行番号を示している」

とあります。そこまでは 「行番号」 はインストラクションポインタのことをわかりやすく書いていただけなのかと思って読んでいました。が、「あくまで行番号」 と強調されていますが、実行時に 「ソースの行番号」 を保持しているのでしょうか？

nakama
12月 28, 2008 10:35 pm
> だどさん

こんにちは。書き込みありがとうございます。

>「インストラクションポインタ（＝現在の実行行番号）は、あくまで行番号を示している」

>とあります。そこまでは 「行番号」 はインストラクションポインタのことをわかりやすく

>書いていただけなのかと思って読んでいました。が、「あくまで行番号」 と強調されて

>いますが、実行時に 「ソースの行番号」 を保持しているのでしょうか？

正確には、「ソースコードの行番号」ではなく、ソースコードがコンパイルされることによって生成されたバイナリである IL コードの行番号、になります。

ただ、IL コード（中間バイナリ）のことを書き始めると話がややこしくなるので、ここでは

ある程度まるめて「ソースコード」と書くことにしました。

ここで「あくまで行番号」と書いたのは、インストラクションポインタが指し示すコードが

インスタンスごとにわかれているわけではない、という意図で書いています。

だどさん
12月 29, 2008 5:25 pm
回答ありがとうございます。

やはり初心者向けに解説記事を書くのは難しいものですね。

ちなみに私も以前は MS にいて (サポートの CPR でした)、ダンプ解析とかやってたのですが、理解とずれたので質問させていただいた次第です。特に理解を正すこともなく、ホッとしました。

とあるコンサルタントのつぶやき
3月 30, 2009 9:14 am
というわけでまたしてもかなり日にちが空いてしまいました；。年度末ということもあって仕事が立て込んでいたのですが、ほぼ一段落したので久しぶりにエントリを。どうしてもまとまった話題を書こうとすると時間がかかっちゃいますね．．．．

とあるコンサルタントのつぶやき
4月 1, 2009 9:10 pm
さて、前回のエントリでは、Windows フォーム内部におけるスレッドの構成や、メッセージループの働きなどについて解説しました。中でも重要なこととして、以下のようなキーポイントがありました。 UI スレッド上で、長時間処理を動かしてはならない。

ヒロ
3月 14, 2014 6:34 pm
無粋なツッコミですが以下の説明はちょっと強引ではないでしょうか。

> そもそもなぜスタックメモリは「スタックメモリ」と呼ばれているのか？ それは、このスタックメモリというのがコールスタックを管理しているものだからです。

スタック操作（FILO)に従ってデータを保存するのがスタックメモリですよね。

スタックの説明をせずに、コールスタックだけで言い切られると

初心者相手にスタックメモリを印象づけるためとはいえ、なんだか違和感が・・・

---

o8q
2月 14, 2017 2:10 am
class Xをインスタンスひとつだけ生成、且つ複数スレッドで同時使用すると・・・
MethodBはスレッドセーフでない。
なので、スレッドセーフなのはMethodAのみってことですね。

ようは、スレッドセーフにしたいなら、状態を保持するな。

---

o8q
2月 14, 2017 2:12 am
まちがえた・・・
class Xをインスタンスひとつだけ生成、且つ複数スレッドで同時使用すると・・・
MethodCはスレッドセーフでない。

Comments are closed.

Microsoft
© 2019 Microsoft